

Algorithm Final Üşengeçlere Özel Çalışma Notu ve Soru Bohçası

Bu notlarla çalışıp finale girdik, 70 beklerken ikimiz de 30 aldık..

Prepared by H.M.V, O.K.U

Cut vertices on graph



Articulation points (or cut vertices) in a graph are vertices that, when removed, increase the number of connected components in the graph. Here is a simple algorithm for finding articulation points in a graph:



1. Initialize a counter `time` to 0, and a list `articulation_points` to store the articulation points.
2. For each vertex `v` in the graph, do the following:
 1. Set the `discovered` time of `v` to be `time`, and increment `time` by 1.
 2. Set the `low` time of `v` to be the `discovered` time of `v`.
 3. For each neighbor `w` of `v`, do the following:
 1. If the `discovered` time of `w` is not set, set it to `time` and increment `time` by 1.
 2. Recursively visit `w`.
 3. Update the `low` time of `v` to be the minimum of the `low` time of `v` and the `low` time of `w`.
 4. If the `low` time of `w` is greater than the `discovered` time of `v`, add `v` to the list of articulation points.
 4. If `v` is the root of the DFS tree and has more than one child, add `v` to the list of articulation points.

Here is some sample Python code that implements this algorithm:

Copy code

```
def find_articulation_points(graph):
    time = 0
    articulation_points = []
    discovered = {}
    low = {}

    def dfs(v, parent=None):
        nonlocal time
        discovered[v] = time
        low[v] = time
        time += 1
        children = 0
        for w in graph[v]:
            if w not in discovered:
                children += 1
                dfs(w, v)
                low[v] = min(low[v], low[w])
                if discovered[v] <= low[w] and parent is not None:
                    articulation_points.append(v)
            elif w != parent:
                low[v] = min(low[v], discovered[w])
        if parent is None and children > 1:
            articulation_points.append(v)

    for v in graph:
        if v not in discovered:
            dfs(v)

    return articulation_points
```

rod-cutting problem

The rod-cutting problem is a classic problem in computer science that involves finding the maximum revenue that can be obtained by cutting a rod into smaller pieces and selling the pieces.

In the rod-cutting problem, you are given a rod of a certain length and a list of prices for rods of different lengths. You are allowed to cut the rod into any number of smaller pieces (at least one) and sell the pieces at the given prices. The goal is to maximize the revenue you receive from selling the pieces of the rod.

For example, suppose you have a rod of length 4 and the prices of rods of different lengths are as follows:

- 1 inch: \$1
- 2 inches: \$5
- 3 inches: \$8
- 4 inches: \$9

You could cut the rod into pieces of length 1 and 3 and sell them for a total of \$1 + \$8 = \$9, or you could cut the rod into pieces of length 2 and 2 and sell them for a total of \$5 + \$5 = \$10. In this case, the maximum revenue is \$10.

Here is a dynamic programming solution for the rod-cutting problem in Python. It has a time complexity of $O(n^2)$ and a space complexity of $O(n)$.

 Copy code

```
def rod_cutting(prices, n):
    # Initialize an array to store the maximum revenue at each
    length
    revenue = [0] * (n + 1)

    # Iterate over the lengths from 1 to n
    for i in range(1, n + 1):
        # Set the maximum revenue to the price of the rod at this
        length
        max_revenue = prices[i - 1]
        # Iterate over the possible cuts at this length
        for j in range(1, i):
            # Update the maximum revenue if a cut at this position
            results in more revenue
            max_revenue = max(max_revenue, revenue[j] + revenue[i
            - j])
        # Store the maximum revenue for this length
        revenue[i] = max_revenue
    # Return the maximum revenue for a rod of length n
    return revenue[n]
```

To use this function, you would pass in an array of prices for rods of different lengths and the desired length of the rod. For example:

 Copy code

```
prices = [1, 5, 8, 9, 10, 17, 17, 20, 24, 30]
print(rod_cutting(prices, 4)) # Output: 10
print(rod_cutting(prices, 5)) # Output: 13
```

peak element in array divide conquer

Sure! Here is an example of a divide and conquer algorithm for finding the peak element of an array in Python:

 Copy code

```
def findPeak(arr, low, high, n):
    # Find index of middle element
    mid = low + (high - low)/2
    mid = int(mid)

    # Compare middle element with its neighbours (if neighbours exist)
    if ((mid == 0 or arr[mid-1] <= arr[mid]) and (mid == n-1 or arr[mid+1] <= arr[mid])):
        return mid

    # If middle element is not peak and its left neighbour is greater than it
    # then left half must have a peak element
    elif (mid > 0 and arr[mid-1] > arr[mid]):
        return findPeak(arr, low, (mid - 1), n)

    # If middle element is not peak and its right neighbour is greater than it
    # then right half must have a peak element
    else:
        return findPeak(arr, (mid + 1), high, n)

# A wrapper function that calls findPeak() for the given array
def peakElement(arr, n):
    return findPeak(arr, 0, n-1, n)

# Test array
arr = [1, 3, 20, 4, 1, 0]
n = len(arr)
print("Index of a peak point is", peakElement(arr, n))
```

This algorithm has a time complexity of $O(\log n)$, making it efficient for finding the

1. Divide and conquer algorithm for one dimensional closest pair

```
def closest_pair(numbers):
    # base case: if there are 2 or 3 numbers in the list, just find
    the minimum distance
    if len(numbers) <= 3:
        min_distance = float("inf")
        for i in range(len(numbers)):
            for j in range(i+1, len(numbers)):
                min_distance = min(min_distance, abs(numbers[i] -
numbers[j]))
        return min_distance

    # divide the list in half
    mid = len(numbers) // 2
    left = numbers[:mid]
    right = numbers[mid:]

    # recursively find the minimum distance in each half
    min_distance_left = closest_pair(left)
    min_distance_right = closest_pair(right)

    # find the minimum distance across the two halves
    min_distance = min(min_distance_left, min_distance_right)

    # find the minimum distance between a number in the left half
    and a number in the right half
    midpoint = (numbers[mid - 1] + numbers[mid]) / 2
    min_distance_across = float("inf")
    for i in range(len(left)):
        for j in range(len(right)):
            if abs(left[i] - right[j]) < min_distance_across:
                min_distance_across = abs(left[i] - right[j])

    # return the minimum of the three distances found
    return min(min_distance, min_distance_across)

# test the function
print(closest_pair([1, 3, 5, 7, 9, 11])) # should print 2
print(closest_pair([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])) # should
```

1. Max square shape in matrix by dynamic programming

Sure! Here is a dynamic programming solution for finding the maximum square shape in a matrix in Python:

 Copy code

```
def max_square(matrix):
    rows = len(matrix)
    cols = len(matrix[0])

    # Create a table to store results of subproblems
    dp = [[0 for _ in range(cols)] for _ in range(rows)]

    # Initialize the maximum size to 0
    max_size = 0

    # Fill the table in bottom-up fashion
    for i in range(rows):
        for j in range(cols):
            # If the current cell is a 1
            if matrix[i][j] == 1:
                # If this is the first row or column, set the
                value to 1
                if i == 0 or j == 0:
                    dp[i][j] = 1
                else:
                    # Otherwise, the value is the minimum value of
                    the
                    # cell above, to the left, and above-left,
                    plus 1
                    dp[i][j] = min(dp[i-1][j], dp[i][j-1], dp[i-1]
[j-1]) + 1

                # Update the maximum size if necessary
                max_size = max(max_size, dp[i][j])

    # Return the maximum size
    return max_size
```

Here is an example of how you could use this function:

 Copy code

```
matrix = [[1, 1, 0, 0, 0],
          [1, 1, 1, 1, 1],
          [0, 1, 1, 1, 1],
          [0, 1, 1, 1, 1],
          [0, 1, 1, 1, 1]]

max_square(matrix)
```

This would return 4, because the largest square shape of 1s in the matrix is a 4x4 square.

find local minimum in array with merge sort python

Here's an example of how you could use the divide-and-conquer approach to find the local minimum in an array in Python:

 Copy code

```
def find_local_minimum(arr):
    if len(arr) == 1:
        return arr[0]
    if len(arr) == 2:
        return min(arr[0], arr[1])

    mid = len(arr) // 2
    left_min = find_local_minimum(arr[:mid])
    right_min = find_local_minimum(arr[mid:])

    return min(left_min, right_min)
```

This function works by dividing the array into two smaller arrays and then recursively finding the local minimum in each of those smaller arrays. The local minimum of the entire array is then the minimum of the local minimums of the two smaller arrays.

Note that this approach has a time complexity of $O(n \cdot \log(n))$, so it may not be the most efficient solution for finding a local minimum in a large array. However, it is

maximum contiguous array sum with dynamic programming python

Here is a dynamic programming solution to find the maximum contiguous subarray sum in Python:

 Copy code

```
def max_contiguous_sum(arr):
    # Initialize maximum sum as the first element
    max_sum = arr[0]
    # Initialize current sum as the first element
    curr_sum = arr[0]

    # Iterate over the array, starting at the second element
    for i in range(1, len(arr)):
        # Update the current sum to be the maximum of either the
        # current element or the sum of the current element and the previous
        # current sum
        curr_sum = max(arr[i], curr_sum + arr[i])
        # Update the maximum sum to be the maximum of either the
        # current maximum sum or the current sum
        max_sum = max(max_sum, curr_sum)

    return max_sum
```

This function takes in an array `arr` and returns the maximum contiguous subarray sum.

For example, `max_contiguous_sum([-1, 3, -5, 4, 6, -1, 2, -7, 13, -3])` would return `21`, which is the maximum sum of the subarray `[4, 6, -1, 2, -7, 13]`.

widest path (bottleneck shortest path) problem in graph with greedy method in python

The "widest path" problem is a variant of the shortest path problem, where instead of trying to minimize the distance traveled, you are trying to maximize the minimum edge weight along the path. One way to solve this problem using a greedy approach is to start at the source vertex and follow the edge with the maximum weight until you reach the destination vertex.

Here is an example of how you might implement this algorithm in Python:

 Copy code

```
def widest_path(graph, source, destination):
    # Set the initial maximum edge weight to 0
    max_weight = 0
    # Set the current vertex to the source
    current_vertex = source
    # Create an empty path
    path = []
    # Repeat until we reach the destination
    while current_vertex != destination:
        # Find the next vertex to visit by looking for the
        # edge with the maximum weight from the current vertex
        next_vertex = None
        for vertex, weight in graph[current_vertex]:
            if weight > max_weight:
                max_weight = weight
                next_vertex = vertex
        # Add the next vertex to the path
        path.append(next_vertex)
        # Set the current vertex to the next vertex
        current_vertex = next_vertex
    # Return the final path
    return path
```